

Relazione Progetto Grafo Orientato e Analizzatore RGB

Michele Bolzoni
Matricola 829834
m.bolzoni3@campus.unimib.it

1 Classe graph

La classe rappresenta un **grafo orientato** generico in C++. Un grafo è un insieme di nodi collegati da archi, dove ogni arco è mono-direzionale e collega un nodo di partenza a un nodo di destinazione.

La classe è implementata come un template con due parametri. Il parametro **T** definisce il tipo generico dell'identificativo dei nodi. Il parametro **E** è un funtore di uguaglianza che stabilisce quando due identificativi rappresentano lo stesso nodo.

2 Struttura

Il grafo è rappresentato da un array `_ids` di tipo `T*` che rappresenta i nodi e da una matrice di adiacenza di booleani `_adj` che rappresenta gli archi. La matrice è $N \times N$, dove N è il numero di nodi e la cella (i, j) vale `true` se esiste un arco dal nodo i al nodo j , `false` altrimenti. La matrice è memorizzata come un unico array lineare piuttosto che come matrice bidimensionale. La cella (i, j) si trova alla posizione $i \cdot N + j$ (ordinamento per righe).

3 Implementazione

Per costruire un oggetto **graph** è necessario specificare due tipi:

1. **T**: il tipo dell'identificativo dei nodi.
2. **E**: un funtore di uguaglianza per confrontare due identificativi e stabilire se rappresentano lo stesso nodo.

I metodi come il costruttore di copia, quello di assegnamento (`operator=`) e il distruttore, gestiscono il ciclo di vita dell'oggetto. È definita anche una funzione di swap, che permette lo scambio di stato tra due grafi.

4 Attributi principali

Gli attributi privati della classe sono:

- `_ids`: array degli identificativi dei nodi. La posizione i contiene l'identificativo del nodo di indice i .
- `_adj`: matrice di adiacenza linearizzata. La cella (i, j) si trova alla posizione $i \cdot \text{_size} + j$.
- `_size`: numero di nodi presenti nel grafo. È di tipo unsigned int.

- **_equals**: funtore di uguaglianza tra identificativi, usato da **indexOf** per stabilire l'identità dei nodi.

Nota: Il numero di archi non è memorizzato in un attributo ma calcolato quando richiesto, contando le celle a **true** nella matrice.

5 Funzionalità e metodi principali

Costruzione e distruzione

- **Costruttore di default**: crea un grafo vuoto, con entrambi gli array a nullptr e dimensione zero.
- **Costruttore di copia**: crea una copia profonda di un altro grafo, allocando memoria propria e indipendente.
- **Distruttore**: libera tutta la memoria allocata dai due array.

Interrogazione

- **numNodes**: restituisce il numero di nodi.
- **numEdges**: restituisce il numero di archi, contando le celle a **true**.
- **existsNode**: verifica se un identificativo è presente tra i nodi.
- **existsEdge**: verifica se esiste un arco tra due nodi dati.

Modifica dei nodi

- **addNode**: aggiunge un nodo. La matrice cresce di una riga e una colonna, inizializzate a **false** perché il nuovo nodo non ha ancora archi. Se il nodo esiste già viene lanciata l'eccezione **node_already_exists**.
- **removeNode**: rimuove un nodo. La matrice si riduce di una riga e una colonna, e gli indici dei nodi successivi vengono compattati. Se il nodo non esiste viene lanciata **node_not_found**.

Modifica degli archi

- **addEdge**: aggiunge un arco mono-direzionale tra due nodi. Se l'arco esiste già viene lanciata **edge_already_exists**.
- **removeEdge**: rimuove un arco. Se l'arco non esiste viene lanciata **edge_not_found**.

Nota: **addEdge** e **removeEdge** condividono l'intera logica in un unico metodo privato **changeEdge(src, dst, value)**. Aggiungere significa scrivere **true** e fallire se l'arco c'è già, rimuovere significa scrivere **false** e fallire se l'arco non c'è.

Iteratore

La classe fornisce un **const_iterator** di tipo forward sui nodi. L'iteratore scorre l'array degli identificativi **_ids** e restituisce i nodi in un ordine non significativo.

Altre operazioni

- **Operatore di assegnamento** (**operator=**): sostituisce il contenuto di un grafo con quello di un altro, tramite la tecnica copy-and-swap.

- **Stampa** (`operator«`): stampa gli identificativi dei nodi e la matrice di adiacenza. Il grafo vuoto è gestito come caso speciale.

6 Scelte implementative

Funtore di uguaglianza esterno invece di `operator==`

Il secondo parametro template `E`, un funtore di uguaglianza, permette di non affidarsi all'`operator==` del tipo `T`. La motivazione è che il criterio di identità dei nodi usato dal grafo può dover essere diverso dall'uguaglianza intrinseca del tipo.

Un esempio concreto è nei test automatici. Si consideri un tipo `file` con percorso e contenuto, il cui `operator==` confronta il percorso (l'identità sul filesystem). Se il grafo deve deduplicare i nodi in base al contenuto, il suo criterio di identità è "stesso contenuto", che non coincide con l'`operator==`. Utilizzare un funtore esterno permette di fare entrambe le cose: due file con percorso diverso ma stesso contenuto sono lo stesso nodo per il grafo, ma diversi secondo `operator==`.

Eccezioni

Ogni operazione lancia un'eccezione quando una preconditione è violata. Interrogare, aggiungere o rimuovere un nodo o un arco riferendosi a un nodo inesistente lancia `node_not_found`. Aggiungere un nodo già presente lancia `node_already_exists`. Aggiungere un arco già presente lancia `edge_already_exists` e rimuovere un arco inesistente lancia `edge_not_found`. Chiedere se esiste un arco fra nodi inesistenti è una violazione della preconditione (cioè che entrambi i nodi esistano), quindi viene lanciata un'eccezione piuttosto che rispondere con **false**.

Matrice linearizzata invece di `bool**`

La matrice è un unico blocco `bool*` anziché un array di array `bool**`. Con `bool**` servirebbero $N + 1$ allocazioni separate (una per l'array di puntatori e una per riga) e altrettante deallocazioni annidate, moltiplicando i punti in cui la gestione della memoria può fallire. Il blocco lineare richiede una sola `new[]` e una sola `delete[]`, al prezzo di dover calcolare esplicitamente l'offset dell'arco ($i \cdot N + j$).

Sicurezza rispetto alle eccezioni

I metodi `addNode` e `removeNode` allocano nuovi array prima di sostituire quelli vecchi. Se un'allocazione o una copia fallisce a metà, i buffer temporanei già allocati vengono liberati e l'eccezione viene rilanciata, lasciando il grafo nello stato precedente all'operazione. In questo modo un'operazione o riesce completamente o non ha alcun effetto, evitando leak di memoria.

7 Progetto Qt: analizzatore RGB

Il secondo progetto è un'applicazione Qt che permette l'analisi della composizione RGB di ogni pixel di un'immagine. L'interfaccia, oltre al bottone per caricare il file, mostra l'immagine caricata e due grafici (la distribuzione delle triplette e l'istogramma delle intensità per canale).

Caricamento e visualizzazione dell'immagine

L'immagine viene selezionata tramite `QFileDialog` e caricata con `QImageReader`. L'immagine è mostrata in una `QLabel` come `QPixmap` riscalata a ogni ridimensionamento della finestra, preservando le proporzioni (`KeepAspectRatio`). L'adattività richiesta dalla traccia è ottenuta tramite i layout manager di Qt e al `resizeEvent`, che ridimensionano i widget al variare della dimensione della finestra.

Struttura dati per le triplette

È definita una struct custom denominata Record, formata da un oggetto QColor e da un int count che indica il suo conteggio.

La soluzione combina due strutture:

- Una **QList** di Record che preserva l'ordine di prima apparizione.
- Una **QHash** che mappa il colore all'indice del Record nella lista fa da tabella di lookup.

La chiave della hash è di tipo QColor, la tripletta impacchettata in un intero: essendo un unsigned int, Qt sa già calcolarne l'hash.

Struttura dati per le intensità di canale

Per le intensità dei singoli canali è necessario memorizzare tutte le intensità del range 0-255 in ordine, comprese quelle assenti dall'immagine, che devono comparire con conteggio zero. Per ciascun canale si usa una QList di 256 interi, dove la posizione è l'intensità e il valore è il conteggio. La dimensione fissa garantisce da sola la copertura totale del range e la presenza delle intensità a zero, senza pre-popolamento.

Entrambe le strutture, quella delle triplette e quelle dei canali, sono popolate in un'unica scansione dell'immagine, riga per riga, per non leggere due volte gli stessi pixel.

Visualizzazione dei grafici

I due grafici sono realizzati con Qt Charts. La distribuzione delle triplette è rappresentata da un grafico a torta in cui ogni fetta corrisponde a una tripletta, con ampiezza proporzionale al numero di occorrenze normalizzato sul totale dei pixel e colore pari al colore reale della tripletta. La torta è stata scelta perché è l'unico grafico di Qt Charts che consente di colorare ogni elemento individualmente, tramite QPieSlice::setColor: nel grafico a barre, invece, il colore si applica all'intero insieme di dati e non alla singola barra. La torta mostra così simultaneamente due informazioni, cioè quali colori compongono l'immagine (colore delle fette) e in che proporzione (ampiezza). Al click su una fetta, questa viene evidenziata e i suoi valori RGB con la percentuale di occorrenza vengono mostrati.

L'istogramma delle intensità è rappresentato da tre spezzate, una per canale, con l'intensità sull'asse delle ascisse e la frequenza normalizzata su quello delle ordinate. Le spezzate sono preferite alle barre perché, con 256 valori per canale, una curva continua è più leggibile di 256 barre sovrapposte.

Nota: Le immagini fornite con la traccia sono quantizzate a poche decine di colori distinti, il che rende la torta a fette colorate leggibile. Su fotografie non quantizzate, con decine di migliaia di colori distinti, la rappresentazione adottata non sarebbe ottimale sia in termini di leggibilità che in termini di costo di calcolo.

8 Valgrind e Make (progetto grafo)

Il progetto è compilato con g++ e i flag -Wall e -std=c++11 (l'unico costrutto C++11 usato è nullptr). La compilazione avviene tramite un Makefile.

Tutti i metodi dell'interfaccia pubblica sono testati nel main, anche su tipi definiti dall'utente. I test usano asserzioni e non richiedono interazione da tastiera.

La gestione della memoria è stata verificata con Valgrind.

```

michele@Helicon:~/c++/260721$ make
g++ -Wall -std=c++11 -c main.cpp -o main.o
g++ main.o -o main.exe
michele@Helicon:~/c++/260721$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./main.exe
==295561== Memcheck, a memory error detector
==295561== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==295561== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==295561== Command: ./main.exe
==295561==
===== Test graph.hpp - 260721 =====

[OK] Test addNode
[OK] Test removeNode
[OK] Test addEdge
[OK] Test removeEdge
[OK] Test completo (nodi + archi)
[OK] Test copia profonda
[OK] Test const iterator
[OK] Test custom type city
[OK] Test custom type file

===== Tutti i test superati =====
==295561==
==295561== HEAP SUMMARY:
==295561==    in use at exit: 0 bytes in 0 blocks
==295561==   total heap usage: 162 allocs, 162 frees, 82,239 bytes allocated
==295561==
==295561== All heap blocks were freed -- no leaks are possible
==295561==
==295561== For lists of detected and suppressed errors, rerun with: -s
==295561== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```